Evidencias: despliegue con Docker y Kubernetes

Backend Spring Boot y frontend Flutter Web en un cluster local de Kubernetes (Docker Desktop)

Proyecto	Flutter App + API Spring Boot
Backend	Spring Boot 3 (Java 17), arquitectura hexagonal, JWT, PostgreSQL (Neon)
Frontend	Flutter Web servido con Nginx
Imágenes	springboot-backend:1.0 · flutter-frontend:1.0
Kubernetes	Docker Desktop (contexto docker-desktop, v1.34.3)
Fecha de las capturas	3 de noviembre de 2026
Repositorio	github.com/Leudy99/flutter-springboot

Contenido

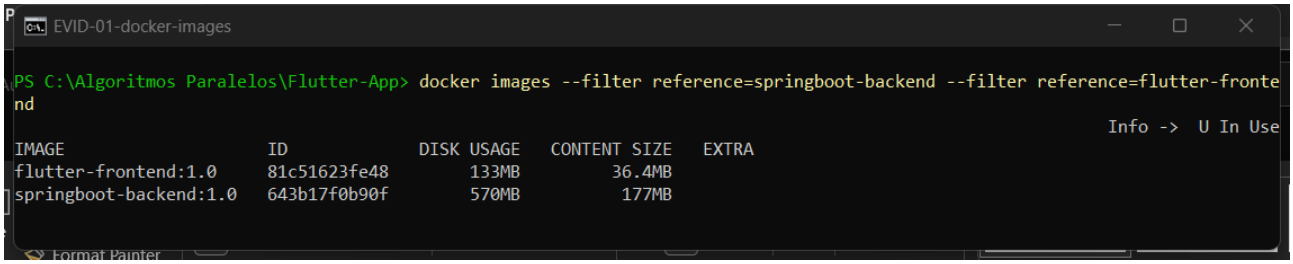
1. Imágenes Docker 2. Pods (backend con 2 réplicas) 3. Deployments 4. Services 5. Aplicación funcionando (frontend y backend) 6. Escalado del backend de 2 a 5 réplicas

Todas las capturas son de la pantalla del equipo, tomadas con los comandos ejecutados contra el cluster. Las credenciales de la base de datos están en un Secret de Kubernetes y no aparecen en los archivos del proyecto ni en estas evidencias.

1. Imágenes Docker

Evidencia 1: Imágenes del backend y del frontend

Comando: `docker images`



Las dos imágenes construidas con los Dockerfile del proyecto: **springboot-backend:1.0** (Spring Boot, multi-stage Maven + JRE 17) y **flutter-frontend:1.0** (Flutter Web compilado y servido con Nginx).

2. Estado inicial: backend con 2 réplicas

Evidencia 2: Pods

Comando: `kubectl get pods`



Dos Pods del backend (**backend-deployment-…**) y un Pod del frontend, todos en estado **Running**.

Evidencia 3: Deployments

Comando: `kubectl get deployments`

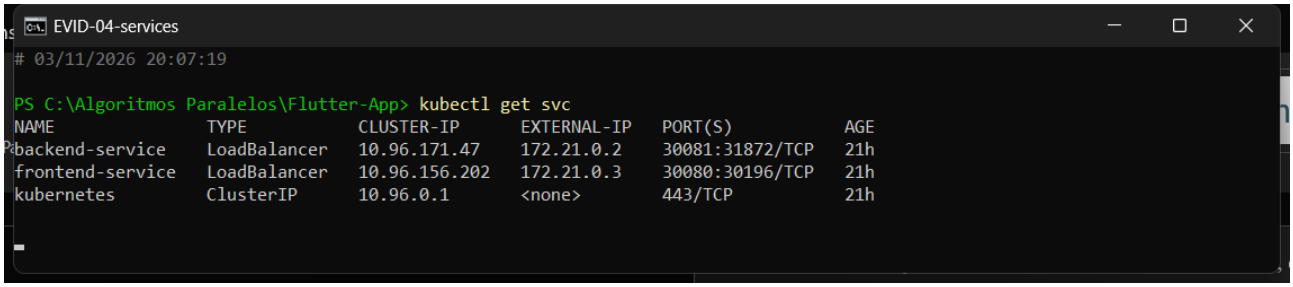


backend-deployment 2/2 (2 réplicas disponibles) y **frontend-deployment 1/1**.

3. Services

Evidencia 4: Services

Comando: `kubectl get svc`

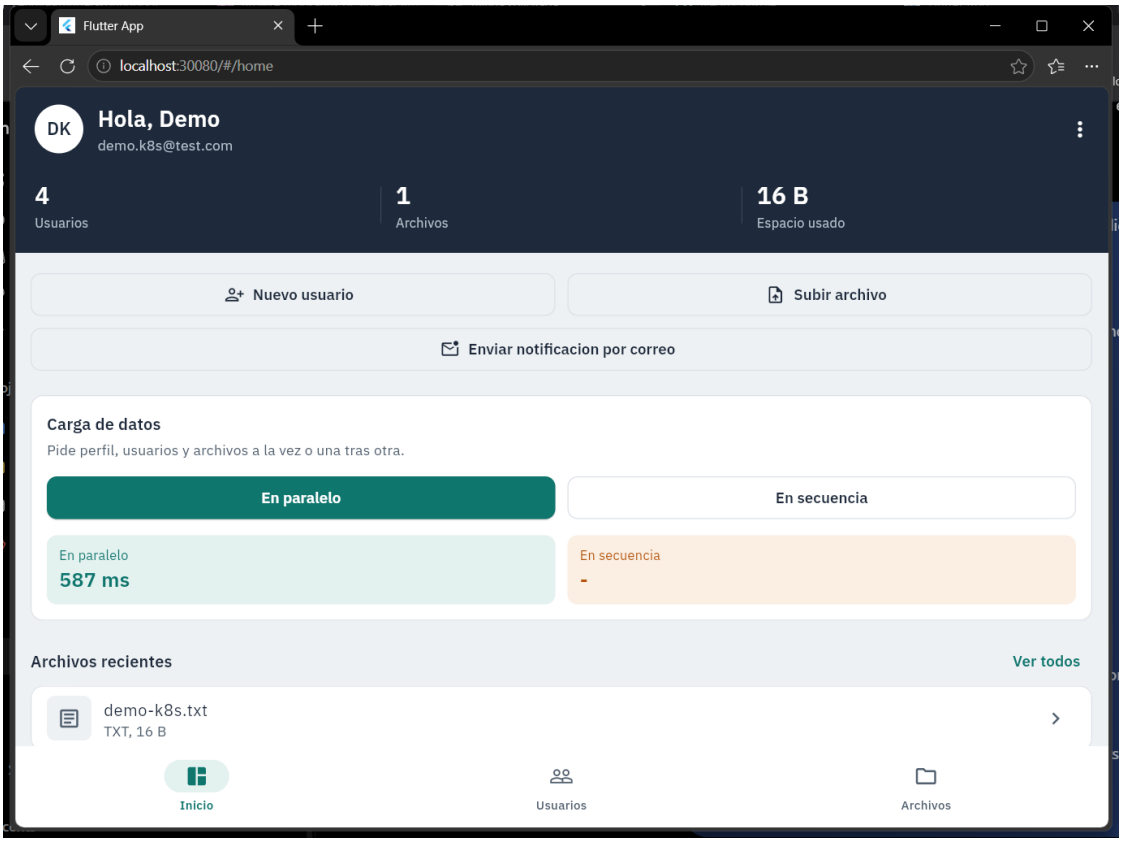


backend-service expone el backend en el puerto 30081 y **frontend-service** expone la aplicación en el puerto 30080; ambos de tipo LoadBalancer, accesibles desde el navegador en localhost.

4. Aplicación funcionando

Evidencia 5: Frontend consumiendo el backend

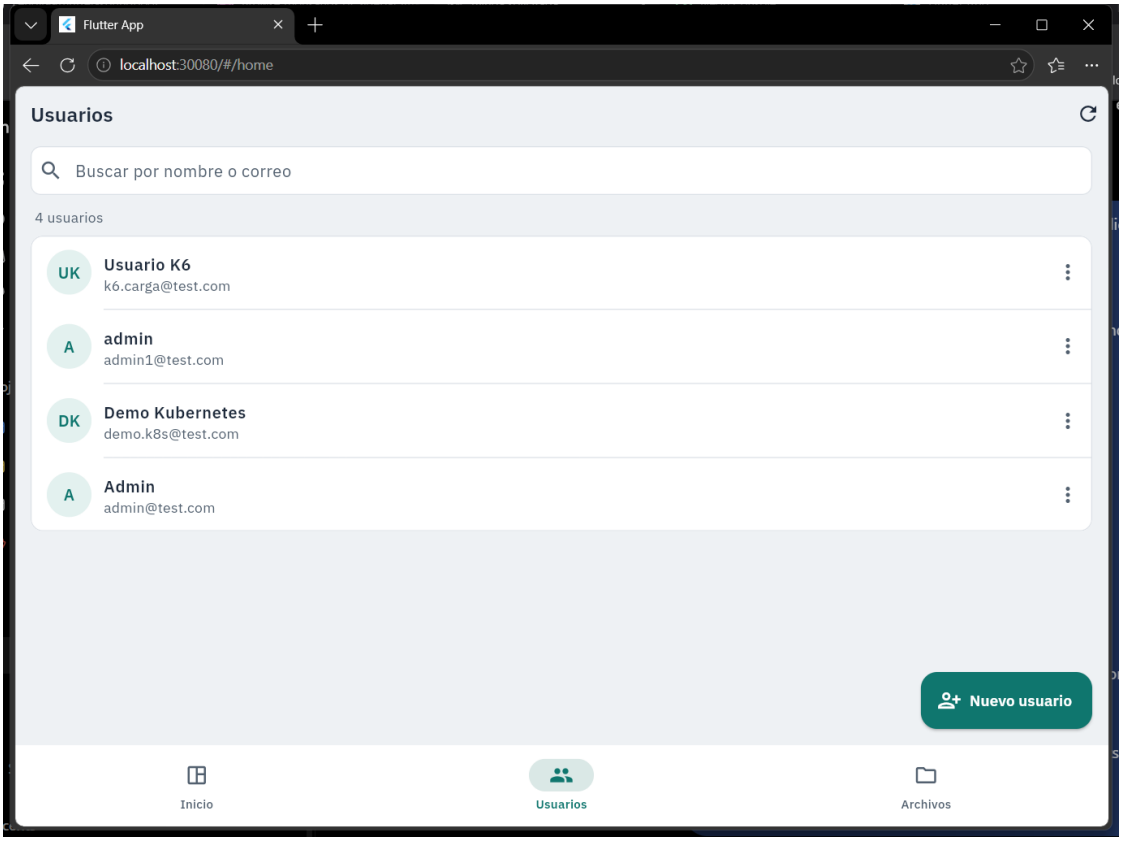
Comando: `http://localhost:30080`



La aplicación Flutter Web servida desde Kubernetes, con la sesión iniciada. Los datos del encabezado (4 usuarios, 1 archivo) y el tiempo de la carga en paralelo (587 ms) vienen del backend desplegado en el cluster.

Evidencia 6: Backend respondiendo: listado de usuarios

Comando: `http://localhost:30080 (pantalla Usuarios)`

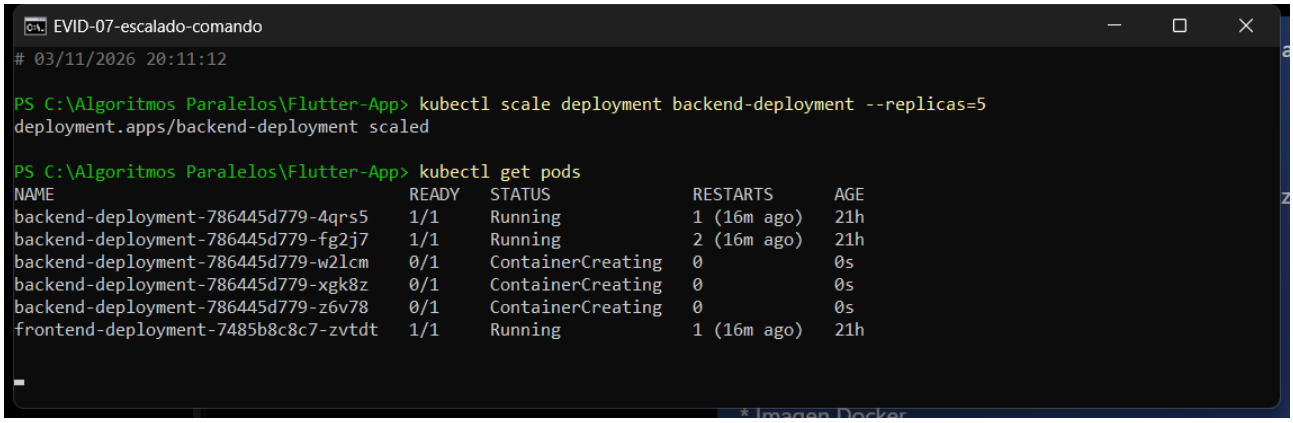


La pantalla de usuarios obtiene los registros mediante la API del backend en Kubernetes (`http://localhost:30081`), que los lee de la base de datos PostgreSQL.

5. Escalado del backend: 2 → 5 réplicas

Evidencia 7: Comando de escalado

Comando: `kubectl scale deployment backend-deployment --replicas=5`



Tras ejecutar el comando, Kubernetes crea tres Pods nuevos (**ContainerCreating**) además de los dos existentes.

Evidencia 8: Deployment con 5 réplicas

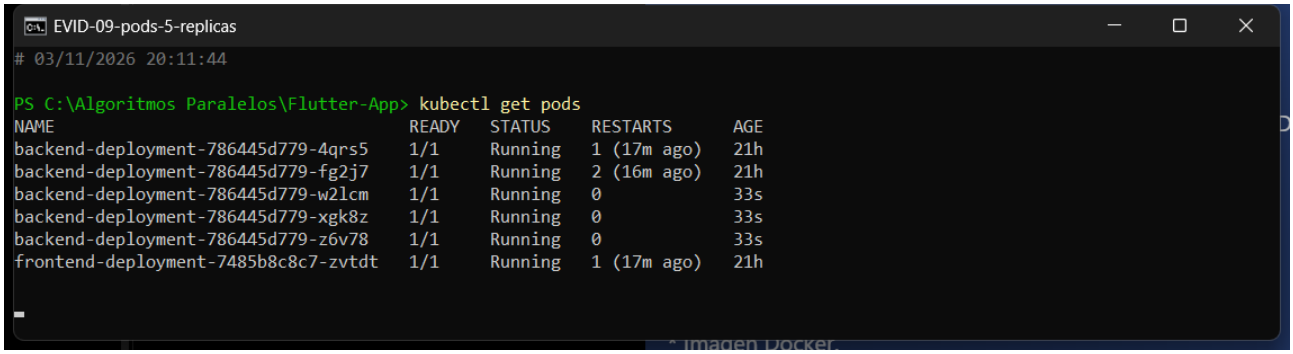
Comando: `kubectl get deployments`



backend-deployment 5/5: las cinco réplicas están listas y disponibles.

Evidencia 9: Los 5 Pods del backend

Comando: `kubectl get pods`



Cinco Pods **backend-deployment-…** en estado **Running**; tres de ellos creados con el escalado (edad 33 s).

Resumen del escalado

	Antes	Después
Réplicas del backend (Deployment)	2	5
Pods backend en Running	2	5
Pods frontend	1	1